

New Methodologies to Solve Hierarchical Reinforcement Learning Problems

Aidan Jared
Master of Data Science
ajar667@aucklanduni.ac.nz

Daoyu Zhang
Master of Data Science

Dominicus Nararya
Master of Data Science
dnar278@aucklanduni.ac.nz

Jason Kong
*Postgraduate Diploma in Computer
Science*
jkon763@aucklanduni.ac.nz

Thomas Clunie
Master of Artificial Intelligence
tclu830@aucklanduni.ac.nz

Yuanyuan Yu
Master of Data Science
yyu119@aucklanduni.ac.nz

Abstract—Traditional Reinforcement Learning techniques struggle with long-term planning and sparse reward environments. This review examines recent Hierarchical Reinforcement Learning literature across five key areas: benchmark environments, model design, reward structures, training methodologies, and evaluation frameworks. Our analysis reveals significant advances in temporal abstraction and reward engineering. The advent of *Gymnasium* has provided a robust and easy-to-use experimental environment that can be leveraged to test Hierarchical Reinforcement Learning hypotheses. The *FeUdal* architecture employs separate controllers for different time scales. A *Manager* sets long-term goals while a *Worker* executes immediate actions to achieve these sub-goals. While *Transformers* excel at long-term credit assignment, they cannot synthesise optimal paths from sub-optimal training data, unlike *Q-Learning* approaches. Rewards drive the Hierarchical Reinforcement Learning process, leading models to success or failure. Maximum-Reward incentivises exploration through maintaining the best reward. *Hindsight Experience Replay* learns from failure to reduce repeated mistakes. The intelligent design of training frameworks by leveraging available resources and splitting the training into separate stages allows for model scaling. Furthermore, the introduction of *rlible* has produced a new standard in Reinforcement Learning Model evaluation by creating statistically sound metrics that reflect true performance without being affected by outliers. These discoveries provide foundational approaches to develop a robust Hierarchical Reinforcement Learning model. However, the large amount of variance in model performance and computational overhead of Hierarchical Reinforcement Learning limits practical deployment. Future research must look into the scalability and reliability of Hierarchical Reinforcement Learning to bring it to its full potential.

Index Terms—Reinforcement Learning, Hierarchical Reinforcement Learning, *Gymnasium*, Atari Learning Environment, *FeUdal* Networks, *Option-Critic*, *Transformers*, Maximum-Reward, Sub-Rewards, Automatic Reward Structures, Two-step RL, Scaling, *rlible*

I. INTRODUCTION

Reinforcement Learning (RL) has long been a training method of models to accomplish human tasks. RL has found success in robotics, self-driving cars, and game playing. With the rise of Large Language Models, RL has found a new role in fine-tuning these architectures. This has sparked research into how RL improves model performance and develops more

generalised strategies. However, most of these successes are in environments with short time horizons or dense reward environments. Traditional Q-learning methods struggle when faced with long-term problems or environments with few rewards.

Hierarchical RL (HRL) is a proposed methodology resolving this issue by dividing RL problems into different hierarchies. Each hierarchy focuses on different aspects of the overall challenge. By having high-level controllers select sub-goals and low-level controllers accomplish the sub-goals, the model can keep track of the whole picture and moment-to-moment choices. For example, in robotics, the high-level controller can keep track of the end goal while the low-level controller can deal with unforeseen issues.

With the complexity of the environments HRL operates in, there are many interconnected research challenges that must be examined. Unlike standard RL, HRL requires careful design of state representations for the different hierarchies to interpret. Well-designed reward structures need to be implemented to drive models to explore and learn. These challenges have sparked research in multiple fronts of model design, reward structure, environments, and model evaluation.

This literature review examines the most recent research and its applications in developing a model that can break down problems into HRL. With specific focuses on Benchmark Environments, Model Design, Reward Design, Model Training, and Performance Evaluation.

II. BENCHMARK ENVIRONMENTS

Unlike most machine learning paradigms, RL lacks standardised environments for comparing methodologies and building upon previous work. To address this challenge, Mark Towers and Jordan Terry designed the *Gymnasium* package [1] as an easy-to-use, reproducible, and robust RL environment for researchers.

Gymnasium's main advantage is its ecosystem of included libraries for model training and comparison. It provides APIs to multiple RL libraries with reliable and scalable implementations. This allows researchers to focus on their research questions rather than implementing baseline models, accelerating research progress. *Gymnasium* provides access

to OpenAI Gym, DeepMind Control Suite, Robotic environments, and the Atari Learning Environment (ALE). It implements environment stepping, parallel RL environments, episode termination and truncation, and both object-oriented and functional APIs. This provides researchers with efficient tools for setting up RL experiments with consistent controls and variables, ensuring reproducible results. These features enable controlled experiments and comparisons to HRL models against established baselines.

ALE enables experiments that require long-term planning while remaining simple to explain. It has also been an evaluation metric for HRL models since its introduction in 2012 [2] due to the game’s difficulty for human players. ALE provides a test bed with clear goal states with long delays between actions and rewards. The sparse rewards, complex navigation, object collection, and high-dimensional state space present a high-end challenge while requiring minimal setup. Games such as *Montezuma’s Revenge*, *Ms. Pac-Man*, and *Seaquest* present different combinations of these challenges, making these games the focus for researchers working on HRL [3], [4], [5]. They are standard benchmarks in HRL research because they require the temporal abstraction and long-term planning capabilities that distinguish hierarchical approaches from flat RL methods.



Fig. 1. *Montezuma’s Revenge*, *Ms. Pac-Man*, and *Seaquest* on Atari

Ms. Pac-Man initially provides frequent rewards through pellet collection, but as levels progress, the increasing speed and complexity create longer intervals between successful actions, effectively creating sparse reward conditions [4]. *Montezuma’s Revenge* presents an almost empty reward environment with long gaps between action and reward, making it very difficult for standard RL models to explore and learn the optimal path [2]. *Seaquest* is a shooting game where players must manage oxygen levels, with points only awarded when returning to the surface with collected items [5]. These games present challenges for standard Q-Learning models, making them valuable benchmarks for RL evaluation. These games specifically challenge HRL in different ways: *Montezuma’s Revenge* tests long-term memory and sub-goal discovery, *Ms. Pac-Man* requires adaptive strategy as game speed and ghost behaviours change, and *Seaquest* demands resource management alongside immediate tactical decisions. Research shows that HRL models outperform standard Deep Q-Networks on these environments through temporal abstraction and improved exploration strategies. Performance measures will be detailed in Section VI with regards to the *rliaile* package.

III. MODEL DESIGN

To approach HRL, models must divide environments into separate problem hierarchies. Research into architectures

that accomplish this divide-and-conquer methodology shows promise by separating models into distinct subsystems.

A. Q-Learning

Traditional RL approaches use Q-Learning algorithms as proposed by Watkins and Dayan [6] to maximise the value function Q . The algorithms learn to select actions a_{t+1} at state s_t to maximise immediate reward r_{t+1} (see Section IV for reward function analysis) as per the equation and the Markov Decision Process (MDP):

$$T(s_t, a_t) = r_t + \gamma \max_a Q(s_{t+1}, a) \quad (1)$$

Through trial and error, Q-Learning learns behaviours that maximise rewards and accomplish tasks. While this architecture demonstrates strong learning ability, it struggles with sparse reward environments and long-term planning.

Hierarchical Reinforcement Learning addresses these shortcomings by decomposing long-horizon tasks into manageable sub-tasks called ‘options.’ This enables agents to reason at different temporal levels: high-level policies focus on long-term goals while low-level policies handle immediate actions. The Option-Critic architecture advances beyond Q-Learning as an HRL framework. It discovers and learns temporally abstract actions called ‘options’ in an end-to-end manner.

B. Option-Critic Architecture

An advancement from Q-Learning is the Option-Critic architecture. As an HRL framework designed to discover and learn temporally abstract actions, known as “options,” in an end-to-end manner. Unlike many hierarchical methods, it requires no manually defined subgoals, intrinsic motivation, or auxiliary rewards. Instead, it learns all option components simultaneously by optimising a single task-level objective. Each option is defined by three learnable components: an intra-option policy (π_ω) that dictates which primitive actions to take, a termination function (β_ω) that determines the probability of the option ending in a given state, and a high-level policy over options (π_Ω) that selects which option to execute. The system is structured as an actor-critic model, where the actor consists of the policies and termination functions, and the “critic” learns the corresponding value functions needed for training.

The model’s key innovation is a pair of policy gradient theorems that enable the simultaneous optimisation of both the internal policies and the termination conditions of all options. The intra-option policy is updated via a gradient that improves its primitive action selections based on their expected value, $Q_U(s, \omega, a)$. The termination function is updated based on a separate gradient derived from the advantage function, $A_\Omega(s, \omega)$. This elegantly links termination to the sub-optimality of the current option; an option is encouraged to terminate in states where its value is lower than the expected value of that state, effectively allowing another, better option to take over. This allows the model to learn how to act within an option, and when to end it, all driven by maximising the total discounted return.

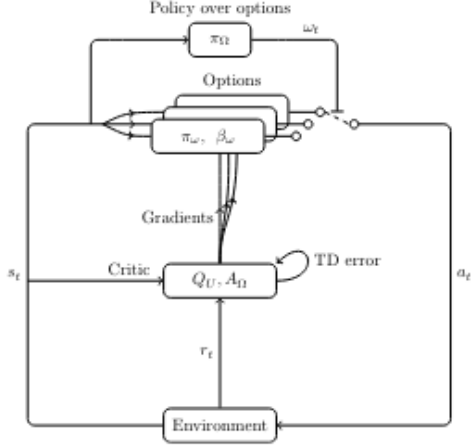


Fig. 2. Option-Critic Architecture from [7]

To apply this design within the Gymnasium package, the Option-Critic agent would interact with a standard environment using its primitive action space. The control flow involves a two-level loop. First, the agent observes a state and uses its high-level policy over options, π_Ω , to select an option ω . It then enters an inner loop where it repeatedly uses the corresponding intra-option policy, π_ω , to choose a primitive action, which is passed to the `env.step()` function. After each primitive action, the agent receives a new state and uses the termination function, β_ω , to stochastically decide whether to break the inner loop and select a new option or to continue with the current one. The stream of (state, action, reward, next state) tuples from the environment is used to update all components of the architecture concurrently: the critic’s value functions and the actor’s intra-option policies and termination functions via their respective gradients. A practical implementation would involve a single neural network with shared initial layers that branch into multiple heads to output the policy over options, as well as the individual policies and termination probabilities for each available option.

Beyond single-stage environments, Reinforcement Learning has also been applied to multi-stage strategy games. For example, research on the Legends of Code and Magic developed an end-to-end policy combined with Optimistic Smooth Fictitious Play to address both deck-building and battle stages. This approach achieved championship performance in the COG2022 competition, highlighting the potential of RL in multi-stage, imperfect-information games [8].

Recent work has shown that incorporating object-level features can significantly improve RL performance. Object-sensitive Deep Reinforcement Learning (O-DRL) introduces object channels to the input representation, allowing agents to distinguish between beneficial and harmful objects. This not only improves performance in Atari games but also provides object saliency maps that make model decisions more interpretable [9].

C. FeUdal Networks

Option-Critic’s success sparked research into further temporal divisions in HRL design. FeUdal Networks, as proposed

by Alexander Sasha Vezhnevets et al. [5], divide the model into a worker and manager to cover moment-to-moment interactions and longer-term planning, respectively. The Manager operates at a lower temporal resolution, akin to a high-level strategist. It observes the environment and sets abstract goals for the Worker to achieve.

Importantly, these goals represent directions in the latent state s_t space rather than absolute states. To learn which directions are advantageous, the Manager is trained using a novel transition policy gradient. This update rule adjusts the goal vector, g_t , to maximise the likelihood of moving towards states that yield higher rewards over a time horizon, c . The gradient for the goal is defined as:

$$\nabla g_t = A_t^M \nabla_\theta d_{\cos}(s_{t+c} - s_t, g_{t(\theta)}) \quad (2)$$

Here, A_t^M represents the Manager’s advantage function (how much better the outcome was than expected), and $d_{\cos}$ is the cosine similarity, which measures how well the Manager’s goal vector g_t aligns with the actual change in the latent state from time t to $t+c$. To reason over these long time horizons, the Manager uses a specialised Dilated LSTM (dLSTM).

The Manager uses a novel RNN design called a dilated LSTM (dLSTM) to effectively reason in long-term horizons. Unlike a standard LSTM, which updates its entire state at every time step, the dLSTM is designed to operate at a lower temporal frequency while still processing every input. The dLSTM’s state is composed of r separate groups of sub-states, or ‘cores’. At any given time step t , only one of these cores is updated, determined by a cyclical schedule ($t_{\text{pmod}r}$). This means each specific core is only updated every r steps, allowing it to preserve its memory over long periods without being constantly overwritten. This “dilation” of updates over time enables the Manager to effectively perform backpropagation through hundreds of steps, making it exceptionally suited for tasks that require long-term credit assignment and memory.

The Worker operates at a much higher temporal resolution, executing primitive actions at every step. It receives the Manager’s directional goal and is trained to act in a way that satisfies this direction. The Worker’s motivation comes from an intrinsic reward, r_t^I , which is calculated based on how well its actions align with the Manager’s recent goals. This reward is defined by the average cosine similarity between the state transitions it achieves and the goals it was given over the dilation period c :

$$r_t^I = \frac{1}{c} \sum_{i=1}^c d_{\cos}(s_t - s_{t-i}, g_{t-i}) \quad (3)$$

This intrinsic motivation is combined with the standard extrinsic reward from the game environment. A key design choice is that gradients are not propagated from the Worker to the Manager. This decoupling prevents the Manager’s goals from becoming simple internal variables and instead forces them to capture meaningful subgoals. The Manager learns what to do, and the Worker learns how to do it. The model demonstrates outperformance on tasks requiring long-term memory and planning, such as the Atari game *Montezuma’s Revenge*, where it autonomously discovered meaningful sub-objectives.

keys k , queries q , and values v , then applying the following equation:

$$z_i = \sum_{j=1}^n \text{softmax} \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right) v_j \quad (7)$$

The attention mechanism learns associations through query-key similarities, theoretically enabling reward credit assignment to previous actions. To capture this relationship, the last K timesteps are fed into the DT. Following the Markov Decision Process framework, states s , actions a , and cumulative rewards R from the last K timesteps are embedded through linear layers, combined with positional encodings for timesteps, then fed into the transformer to predict the next action. The architecture was then tested on Atari and OpenAI benchmarks and compared against the Conservative Q-Learning (CQL) architecture.

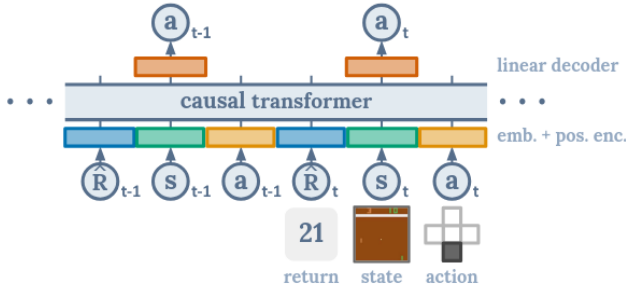


Fig. 4. Decision Transformer architecture from [13]

When trained using imitation learning on labeled trajectories, DT performs similarly to or better than baseline models. It generalises better and can produce returns exceeding those in its training dataset. The context window size K critically affects performance. When $K = 1$, DT performs significantly worse than baselines. Optimal performance occurs around $K \approx 30$, providing sufficient context to identify which policy generated successful actions. “To evaluate this theory, they test the model on a Key-to-Door environment. The Key-to-Door environment consists of three sequential phases: key collection, navigation, and door interaction, with rewards only given upon successful completion of all phases. When compared to the CQL architecture, the DT has a 71.8% success rate on being trained on 1k Random Trajectories, while CQL has a 13.1%. Analysis of the attention weights reveals correlation with pivotal events in the Key-to-Door environment. This helps illustrate the long-term credit assignment that the architecture allows for. The authors conclude that research is needed to see how the DT works in other environments and in real-world settings involving bad actors.

b) Q-Learning Decision Transformer:

The main issue with the DT architecture is a failure to combine parts of sub-optimal paths into an optimal path. Yamagata, Khalil, and Santos-Rodriguez [15] propose combining the advantages of Q-Learning and Decision Transformers.

The authors analyse each model’s strengths and limitations. DT excels at relating scattered information across long sequences and producing accurate outputs. However, it fails

when training data contains sub-optimal paths, unable to synthesise optimal trajectories. Q-learning, on the other hand, can stitch together an optimal path from sub-optimal training data but does not perform well in long-term horizon and sparse reward environments. The authors aim to combine the advantages of these models in what they call the Q-learning Decision Transformer (QDT). CQL generates improved Q-values to replace DT values when they exceed previous state estimates, helping QDT synthesise optimal paths from sub-optimal training data. For example, if $\hat{V}(s_t)$ is the learned value function from the CQL and R_t is the current cumulative reward for DT. If the $\hat{V}(s_t)$ is greater than R_t , then the immediate reward for the previous step is set to $r_{t-1} + \hat{V}(s_t)$; otherwise, it is set as $r_{t-1} + R_t$, which is then repeated till $t = 0$.

By testing this method in the OpenAI gym under different RL tasks and reward densities, the authors find that the QDT operates between the two model architectures. When training on sub-optimal data, QDT performs better than DT, but not as well as the Q-Learning baseline. In sparse reward environments, QDT outperforms Q-learning but underperforms compared to DT. Looking at the performance gap, the authors speculate that it is due to the discount factor γ . After running some tests with a modified γ , they find that a small γ improves model convergence at the expense of long-term horizons and the inverse for a large γ . The authors note that while promising, a new Q-Learning algorithm is needed to enhance the effectiveness of QDT.

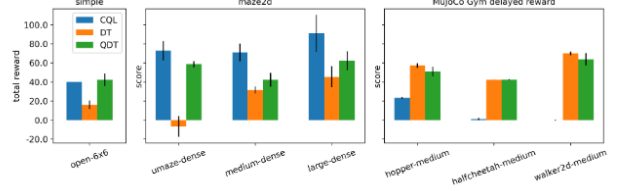


Fig. 5. QDT performance on tasks versus CQL and DT from [15]

When examining Atari games, the moment-to-moment gameplay of moving towards a goal is a task best suited for a traditional Q-Learning model, given its ability to stitch together data from suboptimal paths. While the long-term strategy or complex interaction chains would best be handled by a DT. This demonstrates that these architectures can work together and overcome their mutual weaknesses.

Transformer insights can be integrated with FeUdal architecture designs. The core design of FeUdal (explained in Section III.C) is that a manager works on the long-term goal and a worker on the moment-to-moment choices. The high performance of Transformers in long-term planning makes it a logical upgrade to the standard FeUdal architecture. Following the model design of DT, the previous K sequences are fed into DT and then generate a new goal for the worker. Rewards can be correctly allocated to previous goals, improving long-term planning while having the advantages of Q-Learning. However, Transformers’ large parameter counts create training challenges on standard hardware. Therefore, traditional model architectures have real-world and prototyping advantages.

IV. REWARDS

Rewards drive RL training by incentivising desired behaviour. Without well-structured rewards, models cannot learn their environment effectively. Research into reward system design is as extensive as model architecture research, since reward structures determine how models explore and develop generalised strategies.

A. Maximum-Reward

In RL training, the goal is to learn a policy π that selects actions a_t in states s_t to optimise the cumulative return with immediate reward r_t and discount factor γ as defined by:

$$G_t = \sum_{i=1}^{\infty} \gamma^i r_{t+1+i}, \quad q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t=s, a_t=a] \quad (8)$$

These equations (8) are a solution to the Bellman Equations, which are the foundation of RL. However, this can lead the model to get stuck with suboptimal behaviour. To solve this issue, Vevurko, Bohmer, and Weerdt [16] propose training a model to seek the maximum-reward instead.

Traditional reward paradigms only trigger upon successful task completion, with surrogate rewards guiding agents toward their goals. By modifying the methodology to maximise the reward at each state, the reward functions become simpler, and their tests demonstrate faster convergence to the optimal policy. The authors note that sparse rewards cause agents to get trapped in local optima, since leaving these positions results in reward loss. The authors' proposed solution is maximum-reward, which changes the standard q function as written in (8), to:

$$\hat{q}_{\text{det}}^* = \mathbb{E}_\pi[r_{t+1} \vee \gamma \mathbb{E}_\pi[r(t+1) \vee \dots] | s_t=s, a_t=a] \quad (9)$$

Where $\vee$ is a binary max operation. However, this method only works in deterministic environments when the model does not track previous rewards, which can lead agents to make suboptimal choices when inside a stochastic environment. This is fixed by modifying (8) by expanding and using an approach from stochastic optimal control theory (from [17]) into:

$$\mathbb{E}_\pi[\hat{G}_t] = \mathbb{E}_\pi[r_{t+1} \vee \gamma \hat{G}_{t+1}] \quad (10)$$

Where you can define $y \in \mathbb{R}$ as an auxiliary variable to propagate information between time-steps:

$$\begin{aligned} \hat{q}^\pi(s, a, y) &= \mathbb{E}_\pi[y \vee \hat{G}_{t+1} | s_t=s, a_t=a] \\ y' &:= \frac{R(s, a, s_{t+1} \vee y)}{\gamma} \\ \hat{q}^\pi(s, a, y) &= \gamma \mathbb{E}[y' \vee \hat{q}^\pi(s_{t+1}, a_{t+1}, y') | s_t=s, a_t=a] \end{aligned} \quad (11)$$

Since the smallest immediate reward can be bounded as $r_{t+1} \geq 0$, you can recover the maximum-reward through $y = 0$. By remembering the previous best reward, the model retains exploration tendencies. Since leaving a local optimum yields the same reward as staying, the model maintains exploration incentives. This equal reward structure theoretically encourages continued state space exploration.

This method shows promise with the simplification of the reward structure and the experimental performance the authors cite. Some Atari games present a challenge to traditional reward structures (*Seaquest* and *Montezuma's Revenge*), where the model has long gaps between rewards. The exploration they report makes maximal reward a promising innovation to look at for training an HRL model. Exploration is required to find and overcome obstacles in sparse reward environments. Maximal-Reward would have the model explore the space more, which would help the manager component of the model learn the overall structure of the game.

B. Sub-Rewards

Beyond altering the reward function to maximise returns, another promising approach is to decompose the reward into multiple sub-rewards. The Hybrid Reward Architecture (HRA) proposes learning separate value functions for each component reward and then aggregating them into a single Q-value. This decomposition makes the learning problem easier and more stable, leading to above-human performance in the challenging Ms. Pac-Man environment [18].

C. Advantaged-based Auxiliary Rewards

Tang and Agrawal [19] introduce an HRL approach with Advantage-based Auxiliary Rewards (HAAR). Here, low-level policies are trained with auxiliary rewards that directly arise from the high-level policy's estimate of the advantage function, and are defined as:

$$A(s, a) = Q(s, a) - V(s) \quad (12)$$

The key theoretical contribution is the proof that advantage-based shaping preserves monotonic policy improvement, and hence, joint optimisation at both levels in this hierarchy is stable. On the empirical side, it outperforms both FeUdal and option-critic baselines on MuJoCo continuous control tasks with sparse rewards. It then applies temporal abstraction, annealing, and skill reuse to improve sample efficiency and transfer across tasks. This advantage-driven shaping resolves HRL instability issues and accelerates adaptation in high-variance environments.

D. Automatic Reward Structures

Recent research moves beyond hand-crafted reward structures by developing methodologies that enable agents to generate their own feedback or reinterpret their experiences. As detailed in Dealing with Sparse Rewards in Reinforcement Learning [20], there are four complimentary techniques: curiosity-driven exploration, which rewards the agent for encountering novel or unpredictable states; unsupervised auxiliary tasks, which densify feedback by training the agent on additional unsupervised objectives; random network distillation, which measures novelty via prediction error against a fixed random target; and hindsight experience replay, which transforms unsuccessful episodes into successful experiences by redefining goals retrospectively. Together, these methodologies extend the toolkit for dealing with sparse rewards, enriching the signals available to both high-level and low-level policies in hierarchical Reinforcement Learning frameworks.

a) Curiosity-Driven Exploration:

A recent proposal is Curiosity-Driven exploration by Self-Supervised Prediction [21], which investigates rewarding curiosity to train the model. Their Intrinsic Curiosity Module (ICM) generates an internal reward whenever the agent encounters states that are hard for its predictive model to foresee. In practice, the agent learns an inverse dynamics model and a forward model that predicts the next state’s features; the prediction error (e.g., mean squared error between predicted and actual next-state features) becomes the intrinsic reward. By receiving higher rewards for “surprising” transitions, the agent is driven to explore new states even when extrinsic rewards are absent. This pushes agents to try novel actions, preventing local optima entrapment. One strength of curiosity-based exploration within hierarchical frameworks is that it can help lower-level workers discover useful behaviours or subgoals independently of the high-level reward. A consideration, however, is balancing the intrinsic reward with the top-level task: if not tuned, a curious agent might wander excessively. Still, curiosity enables the hierarchical agent to maintain exploration behaviour and keep exploring until it arrives at actions that eventually yield extrinsic reward, thus improving long-term learning.

b) *Unsupervised Auxiliary Tasks:*

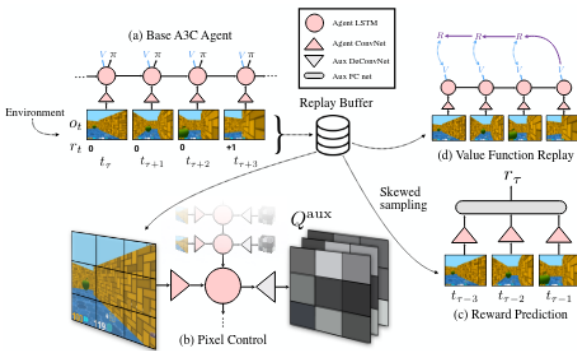


Fig. 6. The UNREAL architecture from [22]

A proposal [22] takes a different approach by supplementing the main objective with additional unsupervised tasks that provide dense learning signals. Their UNREAL framework (Unsupervised Reinforcement and Auxiliary Learning) augments an Asynchronous Advantage Actor-Critic (A3C) agent with auxiliary goals such as pixel control, reward prediction, and value function replay. For instance, the pixel control task trains the agent to maximise changes in visual features (pixels) of the environment, yielding an intrinsic reward for significant changes in the input state. Other auxiliary goals have the agent predict incoming rewards or improve its value function by replaying past experiences. These side tasks do not directly optimise the primary reward; instead, they shape the agent’s representation and encourage exploration of state-space dynamics. The result is a more sample-efficient learner that can extract useful knowledge even when extrinsic rewards are sparse or delayed. In HRL, unsupervised auxiliary objectives can be especially beneficial for the low-level policy, as they force the policy’s neural network to learn richer features and skills (e.g., moving in ways that affect the environment)

that the high-level controller can later exploit. One consideration is to ensure the auxiliary rewards are scaled so as not to dominate the main task; when balanced, they improve stability and exploration, helping the overall hierarchical system learn “good features” and maintain progress in the absence of frequent external rewards.

c) *Random Network Distillation (RND):*

RND, as introduced by Yuri Burda [23] is a simple yet powerful intrinsic reward technique for encouraging exploration in sparse-reward settings. The key idea is to measure state novelty by how unpredictably a fixed, randomly initialised neural network maps that state’s features. RND employs two networks: a frozen target network that produces a random output for each state, and a predictor network that learns to approximate this output. The intrinsic reward is the prediction error of the predictor network on the next state, which is effectively high when the state is unfamiliar (since the predictor hasn’t learned it yet) and low when the state has been visited often. This method is akin to count-based exploration, but it operates in high-dimensional state spaces by learning an implicit state count. Compared to curiosity’s forward dynamics model, RND has the advantage of not getting too good at predicting deterministic environments (since the target is random and non-trivial to extrapolate without visitation). In an HRL context, RND can be seamlessly added as an intrinsic reward for the lower-level policy (or even the high-level policy when selecting subgoals). Its simplicity and low computational overhead make it attractive for complex multi-level agents because the high-level module can trust that the low-level will explore novel states or subgoal outcomes thanks to the RND bonus. A practical strength of RND in HRL is that it reduces the chance that the agent’s worker gets stuck repeating known behaviours, thereby offering the manager a broader palette of state-space experience to formulate long-horizon plans. As with other intrinsic methods, the weight of the RND reward should be tuned so that exploration doesn’t overshadow exploitation of the main task once rewards are found.

d) *Hindsight Experience Replay (HER):*

While intrinsic rewards drive exploration, hindsight techniques address sparse rewards by learning from failed attempts. Advances in Neural Information Processing Systems [24] pioneered HER to cope with environments where the agent might rarely achieve the actual goal during training. The idea is to use each episode, even those where the agent does not reach the goal, as useful data by relabelling the outcome as if it were the intended goal. In other words, whatever final state the agent ends up in is treated as a pseudo-goal that it did accomplish, and the episode is replayed in memory with that goal, allowing the agent to receive reward for it. Over time, this gives the agent experience of “achieving” many different goals and teaches it how to reach a variety of states, ultimately helping it discover strategies to reach the true goal. HER has proven especially crucial for very sparse and high-dimensional tasks (e.g., robotic manipulation or navigation) where random exploration rarely hits the target by chance. In HRL frameworks, HER can be applied at multiple levels: for instance, if a high-level policy sets a subgoal that the low-level fails to

attain, the algorithm can retrospectively treat the low-level’s final state as an alternate subgoal and credit the high-level accordingly. This way, even unsuccessful trajectories yield learning signals for both the meta-controller and the sub-controller. The strength of HER in HRL lies in converting sparse long-term rewards into a denser feedback stream: the higher-level agent learns which subgoals are achievable, and the lower-level agent learns to reach various outcomes, gradually bridging the gap to the real objective. A consideration with HER is that it works best in goal-conditioned settings (where goals can be explicitly specified and substituted). Therefore, the hierarchical algorithm must be designed to handle goal representations. When it is applicable, hindsight replay greatly improves data efficiency and ensures that a hierarchical agent does not waste the information contained in each episode’s outcome, however unintentional that outcome was.

With a suitable model architecture and a well-designed reward structure in place, the final piece of the puzzle is the training methodology itself. The practical implementation of these ideas, from how training is staged to how computational resources are scaled, is crucial for translating theoretical advantages into measurable performance gains.

V. MODEL TRAINING

A. Two-step RL for Multistage Games

learning strategy	random-RL-random	
	winrate [%]	learning time [h]
1-step	48.8	4.0
2-step	72.7	24.0
2-step early-term.	64.2	15.9

learning strategy	RL-RL-random	
	winrate [%]	learning time [h]
1-step	71.4	9.1
2-step	78.5	39.4
2-step early-term.	72.2	19.8

Fig. 7. Performance Comparison of Two Strategies from [25]

Reinforcement Learning traditionally struggles in environments combining randomness, long time horizons, and multi-phase decision-making, where direct training often causes instability or failure. Godlewski and Sawicki [25] proposed a two-step Reinforcement Learning framework for The Lord of the Rings: The Card Game to address this problem. Their method uses curriculum and progressive learning. The agent trains first in simplified settings, then receives fine-tuning throughout the full game. The multi-phase setup specialises actor-critic agents for different decision phases, such as Planning and Questing. This modular design reflects the game’s heterogeneous decision-making structure, preventing a single policy from being overwhelmed by complexity. Experiments confirm that state encoding choices and macro-action abstractions critically affect outcomes, with combined-threat encodings and macro-decisions yielding higher win rates.

Godlewski and Sawicki [25] reported that in the most difficult environment, the success rate of this method reached 78.5%, which was significantly higher than that of Monte Carlo tree search (40%) and evolutionary baseline (60%). These results demonstrate that incremental training and modular design enable RL success in random, complex environments, proving the feasibility of advanced HRL in strategy and card games.

B. Demonstration-Driven Option Discovery

Tamassia et al. introduce learning options from demonstrations alongside reward shaping concepts, applying this approach to a Pac-Man case study [26]. Their algorithm identifies subgoals by detecting moments in time when expert play diverges from the greedy optimal action, inferring the hidden intention. Such subgoals, which have been identified, can be taken later into option theory, with initiation and termination sets. This approach avoids hand-crafted hierarchies while keeping demonstration knowledge explicit within the framework. The results show that agents with options learned through demonstration significantly outperform all baselines and even outperform the very best human design strategy for subtask decomposition, with strong statistical support for long-term returns. The extracted options align with human intuition, such as prioritising power pellets when threatened. This shows that abstraction through demonstration effectively reduces exploration variance.

C. Scaling

Current machine learning paradigms use various scaling approaches, including increasing model size or deploying multiple collaborative agents. Such methods require additional computational power, resulting in studies on getting “compute-optimal” scaling. Compute-Optimal Scaling for Value-Based Deep RL [27] investigates strategies in scaling model size and updates given a budget on compute resources. The authors note how batch size and updates-to-data (UTD) interact to generate a picture of optimally training a scaled RL model.

These methods focus on the UTD ratio (gradient steps per data unit) and model size. Higher UTD ratios require more sequential computations, while larger batches demand more GPU memory. The experiment they set up uses a regularised residual network as the Q values and scales the model width from 256 to 4096 (as discussed in [28]) with batch sizes ranging from 4 to 4096, with other hyperparameters remaining constant, on the DeepMind Control suite and HumanoidBench. The authors first investigate the relationships between model size, batch size, and overfitting. They find that as the model size increases, there is no clear sign of model overfitting, which they speculate is due to RL never fully fitting on the training data. Regarding batch size, it should scale with the model size; the larger the model, the larger the batch size for training. When adding in the UTD ratio, the batch size has an inverse relationship with larger UTD ratios performing best on smaller batch sizes. They propose the following equation to calculate the best batch size for a given model size N and UTD ratio σ :

$$\tilde{B} \approx \frac{a_B}{\sigma^{\alpha_B} + b_B \cdot \sigma^{\alpha_B} \cdot N^{-\beta_b}} \quad (13)$$

This is then combined with the equation for the compute C_J and data efficiency D_J to get the target value J :

$$C_J(\sigma, N) \propto \sigma \cdot N \cdot D_{J(\sigma, N)} \quad (14)$$

Combining these equations (13) (14) leads to the following equations for the UTD ratio and model size given D_0 :

$$\begin{aligned} \sigma^*(D_0) &= \left(\frac{a_\sigma}{D_0 - D^{\min}} \right)^{\alpha_\sigma} \\ N^*(D_0) &= \left(\frac{b_N}{D_0 - D^{\min}} \right)^{\beta_N} \end{aligned} \quad (15)$$

And for a given compute budget F_0 :

$$\begin{aligned} \sigma_F^*(F_0) &= \left(\frac{a_C}{F_0} \right)^{\alpha_c}, \\ N_F^*(F_0) &= \left(\frac{b_C}{F_0} \right)^{\beta_c} \end{aligned} \quad (16)$$

For Hierarchical RL, this research helps identify optimal training parameters for batch size, UTD ratio, and model size. This can help leverage resources into model training and reduce the time spent tuning hyperparameters to get the best performance. In the model architecture under investigation (Section III), there are two interconnected models, which increases the computation needed, so setting up efficient batches, UTDs, and model size is necessary.

VI. MODEL EVALUATION

Model evaluation requires statistically sound measurements. Reinforcement Learning models predominantly use point estimates for performance measurement, but these methods introduce statistical uncertainty due to finite training runs. Agawal, Courville, Bellemare, Castro, and Schwarzer [29] developed *reliable*, which uses bootstrapping techniques to create more accurate RL evaluation metrics from limited training runs. Using the Atari 100k benchmark (training models on 100k gameplay steps across 26 Atari games), they demonstrate current evaluation methods' unreliability. Their package provides statistically sound methods for more accurate RL performance assessment.

Before *reliable*, the standard method of RL model evaluation was point estimates (mean or median) of performance measures across tasks and independent training runs. These metrics create high uncertainty when applied to variable RL performance. While increasing training runs reduces uncertainty, it dramatically increases computational cost—5 runs on 50+ Atari games require 1000 GPU training days. This creates false impressions of model performance, potentially causing researchers to neglect high-performing architectures. The authors of the paper propose three solutions to this issue: interval estimates from stratified bootstrap confidence intervals (CI), performance profiles, and Interquartile Mean (IQM) (performance of the middle 50% of runs) as methods of better assessing RL model performance.

Using a 95% CI means that there is only a 5% chance that the true value does not fall in the interval. Accurate confidence intervals from a few runs N are difficult because bootstrapping assumes that data resampling approximates true distribution sampling. Their solution resamples N runs with replacement for each task M , creating $N \times M$ total samples. Statistics (median, mean, and IQM) are calculated from each sample and repeated to approximate the sampling distribution. Using this method, the authors find they can produce good interval estimates on as few as $N = 10$ for median and IQM.

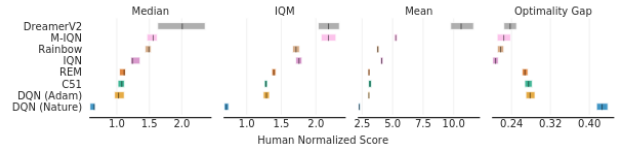


Fig. 8. Aggregate metric produce by *reliable* from [29]

Point estimates, including confidence intervals, obscure important information like heavy tails, outliers, and task variation. Performance Profiles address this by illustrating complete model performance. The distribution proposed is the score distribution, which uses the idea of normalised scores denoted as $x_{1:M,1:N}$, which is evaluated on M tasks for N independent runs. The task scores are then rescaled on two reference points (e.g, 0 for a random agent, 1 for an average human). This score is then added to the following distribution equation:

$$\hat{F}_X(\tau) = \frac{1}{M} \sum_{m=1}^M \frac{1}{N} \sum_{n=1}^N \mathbb{I}[x_{m,n} > \tau] \quad (17)$$

This produces a distribution of scores exceeding scale factor τ . The *reliable* package generates this distribution with confidence intervals, model medians, and mean performance calculated as the area under the curve. This then provides a way to assess the whole model's performance while including data not included in simple CIs.

The last method proposed is using more robust aggregate metrics than median or mean to assess performance at a glance. Using IQM, which discards the bottom and top 25% of runs, then calculates the mean. Median values can overstate performance if there is a high amount of low scores, while outliers influence the mean. In comparison, IQM degrades with lots of low scores and is robust to outliers. This can be tied with the Optimality Gap, a measure of how much the algorithm falls short of a minimum score γ .

reliable can help measure the different performances between HRL models and baseline RL models. This can help get a clear picture of whether models are statistically better than the baselines. However, *reliable* alone cannot determine whether models learned long-term strategies, requiring additional investigation of model weights and task accomplishments.

VII. DOMAIN-SPECIFIC APPLICATION



Fig. 9. Model leaning to play StarCraft 2

A. HRL for RTS Games

Real-Time Strategy games serve as test beds for Hierarchical RL. These games require combining moment-to-moment actions (unit instructions) with long-term strategy, making hierarchical learning essential for effective play.

HRL in RTS Games [30] uses divide-and-conquer strategies similar to FeUdal architecture [5], separating high-level strategic decision-making from low-level execution to make learning more manageable. The architecture consists of a high-level learning component that selects abstract goals and a low-level component that executes them.

A Multi-Layer Perceptron (MLP) serves as the high-level planner, operating over longer timescales. Rather than selecting primitive actions each frame, it chooses from four high-level behaviours: evasive, invade, defensive invade, hunt, and defend base. This MLP receives a curated set of 14 higher-order inputs that represent the game state, including unit-specific information, hit-points, and proximity to bases, as well as game-wide information like the composition of friendly and enemy forces. Each of the MLP’s four outputs represents the predicted Q-value for selecting a corresponding behaviour. The model learns the optimal policy by updating the MLP’s weights using a Q-Learning algorithm. The rewards themselves are event-based, with large values for winning (+1000) or losing (−1000), moderate values for killing an enemy (+100) or dying (−100), and a small negative penalty per time-step (−1) to encourage efficiency.

The low-level execution of these commands is offloaded to pre-programmed Finite State Machines (FSMs). When the high-level MLP selects a behaviour such as “Hunt”, a dedicated FSM takes control of the unit to carry out the specific sequence of movements and attacks required to pursue and destroy the target. This macro-action continues for multiple time steps until a termination condition is met, such as the target being removed or the unit itself dying. This implementation of actions that last for a variable amount of time is a core feature of Semi-Markov Decision Processes (SMDPs), which are well-suited for HRL.

This hierarchical framework is highly adaptable and can be generalised to other complex game genres by separating high-level strategic goals from low-level mechanical execution. The

core principle remains the same: the learning agent’s action space is abstracted from primitive movements to meaningful, long-term “behaviours”. For instance, in a tactical shooter, the MLP’s outputs could correspond to strategies like “attack objective A” or “defend a chokepoint”. In a Multiplayer Online Battle Arena, they might represent goals like “farm a lane for resources” or “contest a map objective.”

Once the high-level MLP selects a strategic behaviour, its implementation is offloaded to simpler, pre-programmed modules or FSMs that handle the moment-to-moment mechanics. These low-level executors would manage tasks like pathfinding to an objective, executing a combat ability rotation in an RPG, or aiming at enemies in a shooter. The strategic agent makes its decisions based on a curated vector of higher-order game state information, such as player health, enemy positions from a minimap, or objective timers, rather than raw visual data. This division of labour simplifies learning by focusing agents on long-term strategy while delegating real-time execution complexities to specialised controllers.

B. Hierarchical Control in Large-Scale Games

Pang et al. tackled the problem of making HRL work for the full StarCraft II game [31]. Their architecture consists of two levels: a controller sub-policy hierarchy, where a high-level controller allocates decision-making to specialised sub-policies, each operating over a reduced macro-action space. Macro-actions were mined from expert replays, compressing the vast primitive action space by one order of magnitude. To train this architecture, Pang et al. employed Proximal Policy Optimisation (PPO), a widely used policy gradient algorithm. PPO optimises policies with the clipped surrogate objective:

$$L^{\text{clib}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (18)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the advantage. The PPO training was provided with appropriate reward shaping and curriculum learning against increasingly strong opponents. The resulting agent achieved a 99% win rate against the easiest AI and a 93% win rate against the hardest non-cheating AI. This shows the powerful combination of hierarchical abstraction, modular sub-policies, and staged learning. This further expands the implications of this paper for developing scalable HRL to complex, high-variance domains, where careful attention to designing macro-actions and curriculum strategies for real-world applications is also required.

VIII. CONCLUSION

This survey reveals a field actively advancing long-term decision-making capabilities. Progress occurs across multiple interconnected fronts rather than following a single path. Foundational architectures like FeUdal Networks and Option-Critic undergo refinement with more efficient learning methods, while Transformers offer alternative paradigms for credit assignment. These architectural innovations are increasingly coupled with sophisticated reward structures—from intrinsic curiosity to hindsight replay—that provide the dense learning signals needed in sparse environments. Growing model com-

plexity makes scalable training methodologies and statistically sound evaluation frameworks like *rlable* increasingly critical. Together, these parallel streams of research are paving the way for agents capable of solving increasingly complex, long-horizon problems.

A. Key Findings and Analysis:

A primary finding is that there is no one-size-fits-all solution; the choice of architecture depends heavily on the problem’s structure.

- FeUdal and explicit HRL architectures suit tasks with naturally decomposable structure, such as RTS games where high-level strategies (‘attack base’) separate clearly from low-level execution (‘move unit to coordinate’). Their explicit hierarchy offers greater interpretability and can be more sample-efficient if the task structure is well-matched.
- Transformer-based models, by contrast, excel where the long-term dependencies are subtle and a clear hierarchy is not apparent. By treating the entire history as a sequence, they can implicitly learn the credit assignment between distant events without needing predefined sub-goals. This makes them powerful in puzzle-like environments like *Montezuma’s Revenge*, though they require more data and computation.
- The choice of reward mechanism is similarly dependent on context. For pure exploration tasks with undefined goals, intrinsic motivation methods like Curiosity-Driven Exploration or Random Network Distillation are highly effective. For goal-oriented tasks in sparse environments, Hindsight Experience Replay is a superior choice as it directly teaches the agent how to achieve a variety of outcomes.

B. Directions for Further Research:

Several promising research directions emerge from this review. Key opportunities include hybrid models combining different approaches, such as integrating Transformers as high-level managers in FeUdal-like frameworks to leverage both explicit structure and sequence modeling. Automatic sub-goal discovery research remains critical. Methods that learn hierarchical task structure from data rather than requiring pre-specification would represent significant advances. Finally, as these models are applied to more complex, real-world domains beyond simulations, research must address the challenges of safety, scalability, and robustness in high-variance, partially observable environments.

IX. AUTHOR CONTRIBUTION

Aidan Jared worked on the Abstract, Introduction (Section I), Benchmark environments (Section II), model design Q-Learning (Section III.A) and transformer sections (Section III.F), maximal reward (Section IV.A), model scaling (Section V.C), and Model Evaluation (Section VI), the figures, and edited/organised the review. Dominicus Nararya worked on the model design Option Critic (Section III.B), FeUdal Networks (Section III.C), RTS games (Section VII.A),

Conclusion (Section VIII), and edited/organised the review. Jason Kong worked on Efficient HRL (Section III.E), RTS (Section VII.B), Auxiliary Rewards (Section IV.C), and Demonstration-Driven Option Discovery (Section V.B). Thomas Clunie worked on the Automatic Reward Structures (Section IV.D). Yuanyuan Yu contributed to the model design of Efficient HRL (Section III.D) and Two-Step RL for Multi-stage Games (Section V.A)

REFERENCES

- [1] M. Towers *et al.*, ‘Gymnasium: A Standard Interface for Reinforcement Learning Environments’, *arXiv preprint arXiv:2407.17032*, 2024.
- [2] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, ‘The Arcade Learning Environment: An Evaluation Platform for General Agents’, *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, Jun. 2013, doi: 10.1613/jair.3912.
- [3] T. Salimans and R. Chen, ‘Learning Montezuma’s Revenge from a Single Demonstration’, *arXiv preprint arXiv:1812.03381*, 2018, [Online]. Available: <https://arxiv.org/abs/1812.03381>
- [4] S. Greydanus, A. Koul, J. Dodge, and A. Fern, ‘Visualizing and Understanding Atari Agents’, *arXiv preprint arXiv:1711.00138*, 2018, [Online]. Available: <https://arxiv.org/abs/1711.00138>
- [5] A. S. Vezhnevets *et al.*, ‘FeUdal Networks for Hierarchical Reinforcement Learning’, *arXiv preprint arXiv:1703.01161*, 2017, [Online]. Available: <https://arxiv.org/abs/1703.01161>
- [6] C. J. C. H. Watkins and P. Dayan, ‘Q-learning’, *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992, doi: 10.1007/BF00992698.
- [7] P.-L. Bacon, J. Harb, and D. Precup, ‘The Option-Critic Architecture’, *arXiv preprint arXiv:1609.05140*, 2016, [Online]. Available: <https://arxiv.org/abs/1609.05140>
- [8] X. Wei *et al.*, ‘Mastering Strategy Card Game (Legends of Code and Magic) via End-to-End Policy and Optimistic Smooth Fictitious Play’, in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.04096>
- [9] Y. Li, K. Sycara, and R. Iyer, ‘Object-sensitive Deep Reinforcement Learning’, in *Proc. Int. Conf. Machine Learning (ICML)*, 2018. [Online]. Available: <https://arxiv.org/abs/1809.06064>
- [10] T. D. Kulkarni, K. R. Narasimhan, A. Saeedi, and J. B. Tenenbaum, ‘Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation’, *arXiv preprint arXiv:1604.06057*, 2016, [Online]. Available: <https://arxiv.org/abs/1604.06057>
- [11] O. Nachum, S. Gu, H. Lee, and S. Levine, ‘Data-Efficient Hierarchical Reinforcement Learning’, *arXiv preprint arXiv:1805.08296*, 2018, [Online]. Available: <https://arxiv.org/abs/1805.08296>
- [12] Z. Wen, D. Precup, M. Ibrahim, A. Barreto, B. Van Roy, and S. Singh, ‘On Efficiency in Hierarchical Reinforcement Learning’, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., Curran Associates, Inc., 2020, pp. 6708–6718. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/4a5cfa9281924139db466a8a19291aff-Paper.pdf
- [13] L. Chen *et al.*, ‘Decision Transformer: Reinforcement Learning via Sequence Modeling’, *arXiv preprint arXiv:2106.01345*, 2021, [Online]. Available: <https://arxiv.org/abs/2106.01345>
- [14] A. Vaswani *et al.*, ‘Attention Is All You Need’, *arXiv preprint arXiv:1706.03762*, 2023, [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [15] T. Yamagata, A. Khalil, and R. Santos-Rodríguez, ‘Q-learning Decision Transformer: Leveraging Dynamic Programming for Conditional Sequence Modelling in Offline RL’, *arXiv preprint arXiv:2209.03993*, 2023, [Online]. Available: <https://arxiv.org/abs/2209.03993>
- [16] G. Vevriurko, W. Böhmer, and M. de Weert, ‘To the Max: Reinventing Reward in Reinforcement Learning’, *arXiv preprint arXiv:2402.01361*, 2024.
- [17] A. Kröner, A. Picarelli, and H. Zidani, ‘Infinite Horizon Stochastic Optimal Control Problems with Running Maximum Cost’, *SIAM Journal on Control and Optimization*, vol. 56, no. 5, pp. 3296–3319, 2018, doi: 10.1137/17M115253X.
- [18] H. van Seijen, M. Fatemi, J. Romoff, R. Larochelle, T. Barnes, and J. Tsang, ‘Hybrid Reward Architecture for Reinforcement Learning’, in

- Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.04208>
- [19] S. Li, R. Wang, M. Tang, and C. Zhang, 'Hierarchical Reinforcement Learning with Advantage-Based Auxiliary Rewards', *arXiv preprint arXiv:1910.04450*, 2019, [Online]. Available: <https://arxiv.org/abs/1910.04450>
 - [20] J. Hare, 'Dealing with Sparse Rewards in Reinforcement Learning', *arXiv preprint arXiv:1910.09281*, 2019, [Online]. Available: <https://arxiv.org/abs/1910.09281>
 - [21] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, 'Curiosity-driven exploration by self-supervised prediction', in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 2778–2787.
 - [22] M. Jaderberg *et al.*, 'Reinforcement Learning with Unsupervised Auxiliary Tasks', 2016, [Online]. Available: <https://arxiv.org/abs/1611.05397>
 - [23] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, 'Exploration by Random Network Distillation', *arXiv preprint arXiv:1810.12894*, 2018, [Online]. Available: <https://arxiv.org/abs/1810.12894>
 - [24] M. Andrychowicz *et al.*, 'Hindsight Experience Replay', in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Curran Associates, Inc., 2017, p. . [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/453fadbd8a1a3af50a9df4df899537b5-Paper.pdf
 - [25] K. Godlewski and B. Sawicki, 'Two-Step Reinforcement Learning for Multistage Strategy Card Game', *arXiv preprint arXiv:2311.17305*, 2023, [Online]. Available: <https://arxiv.org/abs/2311.17305>
 - [26] M. Tamassia, F. Zambetta, W. L. Raffe, F. Mueller, and X. Li, 'Learning Options From Demonstrations: A Pac-Man Case Study', *IEEE Transactions on Games*, vol. 10, no. 1, pp. 91–96, 2018, doi: 10.1109/TCIAIG.2017.2658659.
 - [27] P. Fu *et al.*, 'Compute-Optimal Scaling for Value-Based Deep RL', 2025, [Online]. Available: <https://arxiv.org/abs/2508.14881>
 - [28] M. Nauman, M. Ostaszewski, K. Jankowski, P. Miłoś, and M. Cygan, 'Bigger, Regularized, Optimistic: scaling for compute and sample-efficient continuous control', *arXiv preprint arXiv:2405.16158*, 2024, [Online]. Available: <https://arxiv.org/abs/2405.16158>
 - [29] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, 'Deep Reinforcement Learning at the Edge of the Statistical Precipice', *CoRR*, 2021, [Online]. Available: <https://arxiv.org/abs/2108.13264>
 - [30] R. Niel, J. Krebbers, M. M. Drugan, and M. A. Wiering, 'Hierarchical Reinforcement Learning for Real-Time Strategy Games', in *Proc. 10th Int. Conf. Agents and Artificial Intelligence (ICAART)*, SciTePress, 2018, pp. 470–477. doi: 10.5220/0006593804700477.
 - [31] Z.-J. Pang, R.-Z. Liu, Z.-Y. Meng, Y. Zhang, Y. Yu, and T. Lu, 'On Reinforcement Learning for Full-length Game of StarCraft', *arXiv preprint arXiv:1809.09095*, 2019, [Online]. Available: <https://arxiv.org/abs/1809.09095>